

Temporal Fusion Transformer Based Vertical Scaling Management for Kubernetes

Kemalcan Bora
Barcelona Supercomputing Center
Barcelona, Spain
kemalcan.bora@bsc.es

Elli Kartsakli
Barcelona Supercomputing Center
Barcelona, Spain
elli.kartsakli@bsc.es

Eduardo Quiñones Moreno
Barcelona Supercomputing Center
Barcelona, Spain
eduardo.quinones@bsc.es

Abstract—Language Models (LMs) deployed in Kubernetes face resource management challenges due to their variable computational demands. Traditional scaling methods, like the default Vertical Pod Autoscaler (VPA), struggle to adapt, often causing inefficient resource use and performance issues. We propose the Temporal Fusion Transformer Based Dynamic Vertical Pod Autoscaler (TFT-DVPA), which employs advanced time series forecasting to predict and adjust memory allocation. Tested with three Small Language Models (under 100 million parameters) under simulated fluctuating workloads, TFT-DVPA outperforms the default VPA, achieving lower prediction errors and improved resource efficiency.

Index Terms—Kubernetes, Vertical Scaling, Language Models, Temporal Fusion Transformer, Memory Resource Optimization

I. INTRODUCTION

This work highlights the potential of machine learning driven predictive scaling to address the complexities of managing modern AI workloads in containerized systems, providing a scalable and efficient solution for LM deployments. Language Models (LMs) have gained significant importance in modern applications due to their ability to understand and generate natural language. These models enable advancements across multiple industries, including healthcare, media, manufacturing, and more, by enhancing communication, text analysis, and content creation, making them indispensable in modern technological applications. However, these models require significant computational resources, complicating deployment in resource limited settings like edge devices [1] or cost sensitive cloud environments common among small enterprises.

Recent significant developments have made it possible to use large language models on systems with limited hardware resources. Quantized Low-Rank Adaptation (QLoRA) [2] and Rank Adaptation (LoRA) [3] techniques provide important advantages by reducing model sizes and making their training more efficient. These advancements enable large language models (e.g., LLAMA-3.2 with 8 billion parameters [4]) to run on systems with constrained hardware, which would typically require high-performance computing resources. However, these models adjust with a limited, fixed parameter set, restricting their ability to capture diverse data patterns. Because of this restriction, they may struggle to understand complex relationships, especially over long sequences of in-

formation. This may limit their performance on complex tasks, especially in these resource limited setting [3].

On the other hand, Small Language Models (SLMs) [5] with 77 million to 3 billion parameters [6] offer a solution for problems that negatively affect user experience such as efficiency, latency, security and privacy. Building on the advantages of smaller models, optimized architectures such as Apple Foundation Model, MobileLLM [7], EdgeLLM [8] enable deployment on computing constrained devices while maintaining strong contextual understanding [9].

However, as user adoption increases and applications diversify, both LLMs and SLMs must serve varying numbers of users with fluctuating request patterns. Scaling resources to accommodate these dynamic workloads remains a critical challenge in production environments, where demand can fluctuate unpredictably [10].

Kubernetes (K8s) is the de facto standard for container orchestration in cloud environments and is also being introduced in edge device environments, offering tools for automating deployment, scaling, and management of dynamic workloads [11], [12]. A critical feature is its support for vertical scaling [13], which adjusts resource allocations (e.g., CPU and memory) to pods based on demands. However, a challenge in vertical scaling is the uncertainty of resource requirements. Without precise knowledge of future demands, scaling often relies on heuristic or threshold based approaches, leading to under-provisioning (causing performance degradation) or over-provisioning (increasing costs). This is particularly evident in workloads with variable demands, such as language model based workloads.

To address this challenge, our work focuses on optimizing vertical scaling decisions for memory management in Kubernetes by combining time series forecasting with adaptive resource management. We propose a predictive scaling mechanism utilizing Temporal Fusion Transformers (TFTs), an advanced deep learning architecture introduced by Lim et al. [14]. TFTs are an advanced architecture designed for multi-horizon forecasting, capable of capturing complex temporal patterns and integrating static metadata. By leveraging TFTs, our approach seeks to enhance memory resource allocation efficiency.

In this paper, we first explore related research in Section II to contextualize our work. In Section III, we present a

methodology on what data to collect to train Temporal Fusion Transformers, how to train the TFT, and how to integrate it into Kubernetes as an alternative to default Vertical Autoscaling. In Section IV, we detail our experiment, which involves estimating the RAM consumption of three language models, each with fewer than 100 million parameters, under user traffic simulated using a Fibonacci sequence. The TFT model is employed to predict resource needs based on these traffic patterns. Lastly, in Section V, we conclude with a discussion of our findings and potential directions for future work.

II. RELATED WORK

Recent advancements in autoscaling within cloud-native environments have emphasized vertical and horizontal scaling to enhance resource utilization and minimize latency. Kubernetes, a leading orchestration platform, employs the Vertical Pod Autoscaler (VPA) to adjust CPU and memory allocations for pods using historical usage data, often depicted as descending histograms [15]. Yet, the VPA has limitations, as it overlooks real time cluster resource availability, often causing pod throttling due to insufficient resources and suboptimal memory adjustments [16]. Meanwhile, the Horizontal Pod Autoscaler (HPA) scales pod numbers based on workload demands but cannot alter resource allocations within individual pods, potentially causing uneven resource distribution [15]. These limitations underscore the difficulties in managing dynamic workloads effectively [17], [18]. Machine learning techniques offer a promising approach by using predictive models to analyze usage patterns and predict future resource needs [19], facilitating more adaptive and efficient autoscaling in cloud-native systems [20], [21].

In hybrid autoscaling research, Vu et al. proposed a predictive framework for containerized applications [22]. This system leverages the Kubernetes API and Bi-LSTM (Bidirectional Long Short-Term Memory) for workload prediction, incorporating components for traffic surge detection and a hybrid scaler for vertical and horizontal scaling decisions. Building on the use of recurrent neural networks, Dogani et al. developed K-AGRUED, a Horizontal Pod Autoscaler framework for web applications in Kubernetes, utilizing an attention based Gated Recurrent Units (GRU) encoder-decoder architecture to predict resource usage multiple steps ahead based on Container Decision Trees (CDT). Their method achieved a notable reduction in prediction error, ranging from 2% to 25% compared to existing techniques, as reported in their study [23]. Advancing beyond recurrent models, Shim et al. introduced an autoscaling framework for Kubernetes using a transformer based model, specifically the Informer, designed for long sequence time series forecasting [24]. Their approach predicts future workloads using historical data and applies these predictions to a custom autoscaler. By comparing the transformer model with traditional models like ARIMA and neural network models such as LSTM and Bi-LSTM on NASA and FIFA workload datasets, they demonstrated superior prediction accuracy, with lower mean squared error and higher R^2 scores. This highlights the transformer’s ability to

capture complex temporal dependencies, making it particularly effective for managing dynamic workloads in cloud-native environments.

The application of reinforcement learning has also shown promise in autoscaling. Chaudhari et al. proposed an attention model based on reinforcement learning for predictive autoscaling in cloud environments [25]. This method adapts to changing workloads with high sample efficiency, ensuring accurate scaling decisions. However, reinforcement learning methods often require significant training data and computational overhead, making real time deployment challenging.

While these machine learning approaches, particularly those involving time series prediction, show promise, challenges persist regarding interpretability and the handling of complex temporal dependencies. To address these challenges, Lim et al. introduced the Temporal Fusion Transformer (TFT), which provides interpretable predictions and effectively models intricate temporal patterns [14]. Recently, Ba et al. developed a predictive framework leveraging TFTs to enhance latency aware resource scaling [26]. Their approach predicts end-to-end latency by analyzing frontend request patterns and resource utilization metrics, identifying potential Service Level Agreement (SLA) violations. When violations are detected, the model’s feature importance outputs are fed into a Kernel Ridge Regression (KRR) algorithm to calculate necessary parameter adjustments, ensuring target latency is maintained. The authors validated this methodology using a microservice based application, demonstrating its practical applicability [26]. In a related application, Luo et al. adapted the TFT architecture for large scale Key Performance Indicator (KPI) anomaly detection in microservice systems [27]. Their framework, TFTOps, incorporates probabilistic inputs to improve prediction accuracy and enables concurrent monitoring of hundreds of time series such as CPU usage, network I/O, and request rates using a single model with shared parameters.

Despite recent advancements in autoscaling mechanisms for cloud-native applications, including transformer based models for predictive scaling and anomaly detection, few studies address highly variable workloads, such as those in LM serving infrastructures. For instance, Shim et al. [24] focus on general web server workloads, while Ba et al. [26] target microservice latency optimization. Although Luo et al. [27] demonstrate TFT’s effectiveness for anomaly detection in microservices, their work does not address the memory intensive demands of LM workloads or vertical scaling. Our work addresses this gap by optimizing vertical scaling in Kubernetes, focusing on memory management for LM workload.

III. PROPOSED METHODOLOGY

Our research optimizes vertical scaling in Kubernetes by leveraging TFTs to predict memory usage from historical data. Our proposed system, the Temporal Fusion Transformer Based Dynamic Vertical Pod Autoscaler (TFT-DVPA), integrates a pre-trained TFT forecasting model with a custom Kubernetes controller to dynamically adjust memory allocations for containerized Language Model workloads. TFTs are particularly

effective at capturing complex patterns like non-linearity and seasonality in the workloads of containerized applications.

The proposed methodology is presented in three parts: (1) an overview of TFTs and their advantages for time-series forecasting, (2) the process for model training, which includes server-side data collection from an LM application under client-side simulated load, and (3) the operational architecture of the TFT-DVPA, detailing how these components interact within a Kubernetes environment to achieve predictive scaling.

These components collectively enable data collection, resource demand forecasting, and dynamic scaling adjustments, improving efficiency over traditional approaches.

1) *Method Description: Temporal Fusion Transformers:* TFTs, introduced by Lim et al. [14], are an advanced deep learning architecture designed for multi-horizon forecasting. In our work, TFTs predict memory usage for containerized applications in Kubernetes using high-frequency historical data. This component includes:

- **TFT Description:** TFTs integrate static metadata with time series data via attention mechanisms, effectively modeling non-linear patterns and seasonality.
- **Benefits:** They offer probabilistic forecasting (e.g., quantile predictions from 1st to 99th percentiles) for uncertainty quantification, and their attention mechanisms enhance accuracy by focusing on relevant temporal features.
- **Suitability:** TFTs are ideal for dynamic Kubernetes environments due to their ability to handle noisy, high-frequency data, enabling preemptive scaling to prevent issues like Out of Memory (OOM) errors.

2) *Model Training and Data Preparation:* This component outlines the process of preparing data and training the TFT model, split into server-side data preparation and client-side load testing for workload simulation.

a) *Server-Side Data Preparation:* The server-side application, implemented as a Flask-based¹ service, hosts text generation models (e.g., google-t5/t5-small, facebook/blenderbot_small-90M, shiontendon/error_diagnostic_model_80M) and monitors resource usage. It includes:

- **Resource Monitoring:** Using the psutil² library, memory usage is logged at one-second intervals to CSV files (e.g., raw_memory_usage_*.csv) via a dedicated thread, ensuring uninterrupted service.
- **Data Preprocessing:** The logged data is then processed in several steps: Timestamps are standardized to a one-second frequency, the data is converted into a TimeSeries object using the Darts library³, and finally, it is normalized using a Scaler transformer to ensure stable TFT training.
- **Service Delivery:** The endpoint /generate_text handles POST requests for text generation, while /memory_stats provides real time usage statistics via GET requests.

This data preparation forms the foundation for accurate forecasting.

TABLE I: Sample of System Resource Usage Metrics

Metric	Value
Timestamp	2025-03-02 12:04:38
Memory Usage (MB)	564.39
Selected Model	google-t5/t5-small

b) *Model Training Process:* After collecting and pre-processing the server-side data, we train the TFT model to forecast memory usage. The preprocessed time series data is temporally split into training (80%) and validation (20%) sets to preserve sequential patterns. The TFT model is configured with input and output window lengths of 25 seconds each, capturing sufficient temporal context to detect memory usage patterns while avoiding irrelevant historical noise. Multiple quantile levels are used for probabilistic forecasting to account for uncertainty in future resource demands. The model is trained for multiple epochs with appropriate batch sizing, incorporating relative time indices to maintain temporal relationships in the data. Performance is continuously monitored on the validation set throughout the training process. After training, the model forecasts memory usage for the next 25 seconds. These predictions are stored in Redis for fast access by the Kubernetes integration, enabling timely vertical scaling. This approach helps the TFT model learn temporal memory patterns, supporting accurate dynamic resource allocation.

c) *Client-Side Load Testing:* To generate realistic workloads, we use the Locust framework⁴ to simulate user interactions:

- **Workload Generation:** Concurrent users scale according to the Fibonacci sequence (up to 144), sending randomized text generation requests (e.g., “explain quantum computing”) to the server.
- **Tasks:** The generate_text task sends POST requests to /generate_text, and check_memory_stats retrieves usage via GET requests to /memory_stats.
- **Pattern:** Each load level runs for 30 seconds, followed by a 30 second stabilization period before incrementing, mimicking increasing traffic.

This workload data is used for both TFT training and autoscaler evaluation.

3) *Simulator Environment: Integrated Pipeline:* The simulator environment integrates all components into a dynamic, closed-loop system:

- **Data Collection:** The server logs memory usage during client driven workloads.
- **Prediction:** The TFT model, trained on preprocessed data, forecasts memory usage for the next 25 seconds, storing results in Redis⁵.
- **Scaling:** A Kubernetes integration component, written in Go⁶, retrieves predictions using go-redis⁷ and updates

¹<https://flask.palletsprojects.com/en/stable/>

²<https://github.com/giampaolo/psutil>

³<https://unit8co.github.io/darts/>

⁴<https://locust.io/>

⁵<https://redis.io/docs/latest/develop>

⁶<https://go.dev/doc/>

⁷<https://redis.io/docs/latest/develop/clients/go/>

the Vertical Pod Autoscaler (VPA) via `client-go`⁸.

IV. EXPERIMENTS

In this section, we present the evaluation of our proposed TFT based vertical scaling approach and the performance assessment of our text generation service under varying load conditions.

A. Experimental Setup

The experiments for evaluating our TFT-based Dynamic Vertical Pod Autoscaler (TFT-DVPA) were conducted in a resource-constrained environment featuring 16 GB of RAM and 8 CPU cores. These specifications were chosen to mirror those of edge computing platforms like the NVIDIA Jetson AGX Xavier⁹, a common target for deploying SLMs due to their efficiency. This CPU-based setup provided a controlled setting in which we could assess the performance and scalability of our memory forecasting and autoscaling mechanisms, particularly for scenarios where dedicated GPU resources might be limited or unavailable. This configuration allowed us to explore the system’s ability to adapt to dynamic memory demands under typical edge or resource limited cloud conditions.

B. Default Vertical Pod Autoscaler

VPA in Kubernetes automatically adjusts the resource requests and limits for containers based on historical usage patterns. It ensures that applications receive enough CPU and memory while avoiding unnecessary excessive allocation.

In our implementation, we configured the VPA with the following settings:

- **API version:** `autoscaling.k8s.io/v1`
- **Kind:** `VerticalPodAutoscaler`
- **Metadata:**
 - Name: `grpc-service-vpa`
- **Update policy:**
 - Update mode: `Auto`
- **Resource policy:**
 - Container policies (applies to all containers):
 - * Minimum allowed: 128Mi memory, 500m CPU
 - * Maximum allowed: 850Mi memory, 4000m CPU
 - * Controlled resources: `cpu, memory`

The maximum memory allocation was set to 850Mi, determined from an observed average consumption of approximately 600Mi, with occasional peaks up to 800Mi. However, under high and unpredictable load conditions during experiments, memory demands exceeded this limit. The VPA, relying solely on historical data, failed to adapt, resulting in Out of Memory (OOM) issues, as shown in the error log in Table II. Given that the selected SLMs have comparable parameter sizes (60M, 80M, 90M), we assumed their impact on consumption would be similar.

TABLE II: Pod Status After OOM Error

Field	Value
Port	50055/TCP
Host Port	0/TCP
Last State	Terminated
Reason	OOMKilled
Exit Code	137
Restart Count	23

The ‘OOMKilled’ error occurs when a container uses more memory than it’s been allocated, causing the system to forcefully shut it down. This shows that the default VPA approach, which relies only on historical averages, isn’t enough to handle sudden spikes in workload. We observed that the VPA automates resource allocation but cannot predict or adjust memory limits for sudden demand spikes.

C. TFT-based Dynamic Vertical Pod Autoscaler

We evaluated three SLMs with fewer than 100 million parameters `google-t5/t5-small` (T5-S, 60M parameters), `facebook/blenderbot_small-90M` (BB-90M, 90M parameters), and `shiontendon/error_diagnostic_model_80M` (EDM-80M, 80M parameters) to assess their efficacy in modeling historical memory usage data. This data, collected at one second intervals, was preprocessed using the Darts library to construct time series sequences. We employed a temporal split of 80% for training and 20% for validation, with normalization applied via a Scaler transformer to ensure consistent scaling across sequences. Hyperparameter configurations were systematically explored, varying LSTM layers, attention heads, and hidden sizes, with validation mean absolute error (`val_mae`) serving as the primary performance metric, where lower values indicate higher predictive accuracy.

The top performing configurations for each model are summarized in Table III. For T5-S, the best configuration featured a hidden size (HS) of 32, 8 LSTM layers (LL), and 4 attention heads (AH), achieving a `val_mae` of 73.17, closely followed by a setup with a hidden size of 64, 8 LSTM layers, and 4 attention heads, yielding a `val_mae` of 79.40. BB-90M excelled with a hidden size of 16, 4 LSTM layers, and 8 attention heads, recording a `val_mae` of 82.26, while its second best configuration (hidden size=32, 4 LSTM layers, 8 attention heads) produced a `val_mae` of 109.77. EDM-80M’s optimal setup included a hidden size of 16, 8 LSTM layers, and 4 attention heads, resulting in a `val_mae` of 198.94, with its second best configuration (hidden size=32, 4 LSTM layers, 2 attention heads) achieving a `val_mae` of 200.86.

Analysis of these results reveals distinct patterns in hyperparameter effectiveness. The combination of 8 LSTM layers and 4 attention heads emerged as a strong configuration, securing the top `val_mae` for T5-S and EDM-80M and performing reasonably well across all models. In contrast, BB-90M favored a lighter architecture with 4 LSTM layers and 8 attention heads, suggesting that its architecture may benefit from broader attention mechanisms over deeper recurrence.

Hidden size, however, showed model specific variability. T5-S achieved optimal performance with a hidden size of 32,

⁸<https://github.com/kubernetes/client-go>

⁹<https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-xavier-series/>

while BB-90M and EDM-80M performed best with a smaller hidden size of 16. This discrepancy suggests that hidden size tuning is highly dependent on model architecture and task complexity, requiring individualized adjustment.

TABLE III: Model Configurations by Val MAE

HS	LL	AH	MAE	Model
32	8	4	73.17	T5-Small (60M) Best
64	8	4	79.40	T5-Small (60M) 2nd
16	4	8	82.26	BlenderBot (90M) Best
32	4	8	109.77	BlenderBot (90M) 2nd
16	8	4	198.94	ShionTendon (80M) Best
32	4	2	200.86	ShionTendon (80M) 2nd

Based on this analysis, the combination of 8 LSTM layers, 16 hidden size and 4 attention heads is recommended as the optimal setup for consistent performance across these models, achieving top results in two and reasonable performance in the third. Using this configuration we compare VPA and TFT-based DVPA.

Figure 1 compares actual memory usage, predicted consumption, and scaling behavior between the default VPA and TFT-based DVPA. For SLMs under 100 million parameters, memory consumption was nearly identical across models, allowing us to use an averaged value (blue line) for simplicity. VPA induced crashes (blue cross markers) occurred when allocated memory failed to handle workload fluctuations.

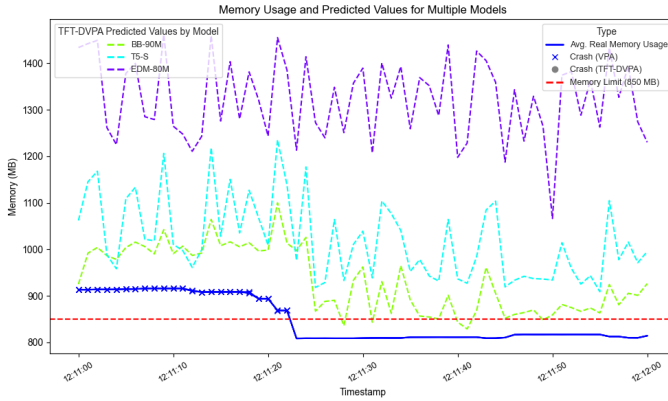


Fig. 1: Comparison of Default VPA and TFT-Based Dynamic VPA in Predicting and Allocating Memory for Small Language Models.

In contrast, the TFT-based DVPA predicts memory demands based on learned temporal patterns and probabilistic forecasting (90th percentile quantile estimation) to predict and preemptively adjust memory allocation, avoiding critical thresholds. These results underscore the limitations of traditional autoscaling mechanisms like the default VPA in managing rapidly changing memory demands of SLM applications. The TFT-based DVPA enhances stability and efficiency, reducing downtime risks and ensuring smoother operation in Kubernetes environments.

V. CONCLUSION AND FUTURE WORKS

In this work, we demonstrate the impact of predictive scaling with the TFT model on memory resource allocation in Kubernetes, particularly for small language model workloads below 100 million parameters. Our findings show that TFT-based scaling effectively manages memory resources, maintaining text generation service stability under fluctuating workloads. While this study excluded CPU usage data, future work will include it as a parameter, given its importance in training processes like backpropagation. Additionally, we aim to test other models with parameter sizes between 2.3 billion and 4 billion, beyond the small language models examined here. Future improvements could focus on hybrid scaling approaches that combine TFT-based prediction with autoscaling for even better resource optimization. To support reproducibility and further research, the dataset and resources used in this study, including the implementation code, are publicly available at GitLab Repository¹⁰. This repository contains the necessary datasets, scripts, and documentation to replicate our experiments and build upon this work.

ACKNOWLEDGMENT

The research leading to these results has received funding from the European Union’s Horizon Europe Programme under the EXTRACT Project, grant agreement n° 101093110 and the PROXIMITY project (PID2021-124122OA-I00/AEI/10.13039/501100011033). This work is co-financed by the ASCENDER project of the UNICO I+D Cloud program that has Ministry for Digital Transformation and of Civil Service and the EU-Next Generation EU as financing entities, within the framework of the PRTR and the MRR. The first author also acknowledges support from the AI4Science Fellowships (AI4S) program. This initiative is part of the talent attraction and retention programs within Investment 4 of Component 19 of the Recovery, Transformation, and Resilience Plan (C005/24-ED CV1).

REFERENCES

- [1] T. Chen, “Challenges and opportunities in integrating llms into continuous integration/continuous deployment (ci/cd) pipelines,” in *2024 5th International Seminar on Artificial Intelligence, Networking and Information Technology (AINIT)*, IEEE, 2024, pp. 364–367.
- [2] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: Efficient finetuning of quantized llms,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [3] E. J. Hu, yelong shen, P. Wallis, *et al.*, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=nZeVKeeFYf9>.

¹⁰<https://gitlab.bsc.es/ppc-bsc/software/tft-dvpa>

- [4] A. Dubey, A. Jauhri, A. Pandey, *et al.*, “The llama 3 herd of models,” *CoRR*, vol. abs/2407.21783, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2407.21783>.
- [5] Q. Zhang, Z. Liu, and S. Pan, “The rise of small language models,” *IEEE Intelligent Systems*, vol. 40, no. 1, pp. 30–37, 2025. DOI: [10.1109/MIS.2024.3517792](https://doi.org/10.1109/MIS.2024.3517792).
- [6] P. Lepagnol, T. Gerald, S. Ghannay, C. Servan, and S. Rosset, “Small language models are good too: An empirical study of zero-shot classification,” in *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, N. Calzolari, M.-Y. Kan, V. Hoste, A. Lenci, S. Sakti, and N. Xue, Eds., Torino, Italia: ELRA and ICCL, May 2024, pp. 14 923–14 936. [Online]. Available: <https://aclanthology.org/2024.lrec-main.1299/>.
- [7] D. Chen, Y. Liu, M. Zhou, *et al.*, “Llm for mobile: An initial roadmap,” *ACM Transactions on Software Engineering and Methodology*, 2024.
- [8] D. Xu, W. Yin, H. Zhang, *et al.*, “Edgellm: Fast on-device llm inference with speculative decoding,” *IEEE Transactions on Mobile Computing*, 2024.
- [9] B. Bokkena, “Optimizing cloud infrastructure management using large language models: A devops perspective,” in *2024 2nd International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS)*, IEEE, 2024, pp. 1401–1406.
- [10] L. Baresi, D. Y. X. Hu, G. Quattrocchi, and L. Terracciano, “Kosmos: Vertical and horizontal resource autoscaling for kubernetes,” in *International Conference on Service-Oriented Computing*, Springer, 2021, pp. 821–829.
- [11] K. Kaur, S. Garg, G. Kaddoum, S. H. Ahmed, and M. Atiquzzaman, “Keids: Kubernetes-based energy and interference driven scheduler for industrial iot in edge-cloud ecosystem,” *IEEE Internet of Things Journal*, vol. 7, no. 5, pp. 4228–4237, 2019.
- [12] T. Goethals, F. De Turck, and B. Volckaert, “Extending kubernetes clusters to low-resource edge devices using virtual kubelets,” *IEEE Transactions on Cloud Computing*, vol. 10, no. 4, pp. 2623–2636, 2020.
- [13] Kubernetes Documentation. “Vertical pod autoscaling.” Accessed: Feb. 3, 2025. (2025), [Online]. Available: <https://cloud.google.com/kubernetes-engine/docs/concepts/verticalpodautoscaler>.
- [14] B. Lim, S. Ö. Arik, N. Loeff, and T. Pfister, “Temporal fusion transformers for interpretable multi-horizon time series forecasting,” *International Journal of Forecasting*, vol. 37, no. 4, pp. 1748–1764, 2021.
- [15] G. Rattihalli, M. Govindaraju, H. Lu, and D. Tiwari, “Exploring potential for non-disruptive vertical auto scaling and resource estimation in kubernetes,” in *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*, IEEE, 2019, pp. 33–40.
- [16] Z. Bouflous, F. Haraka, M. Ouzzif, and K. Bouragba, “Enhanced vertical pod auto scaling with decision tree regressor-max in kubernetes,” in *2024 11th International Conference on Wireless Networks and Mobile Communications (WINCOM)*, IEEE, 2024, pp. 1–5.
- [17] M.-C. Chiang, L.-W. Zhang, Y.-M. Chou, and J. Chou, “Dynamic resource management for machine learning pipeline workloads,” *SN Computer Science*, vol. 4, no. 5, p. 665, 2023.
- [18] M.-C. Chiang and J. Chou, “Dynamoml: Dynamic resource management operators for machine learning workloads,” 2021.
- [19] S. Verreydt, E. H. Beni, E. Truyen, B. Lagaisse, and W. Joosen, “Leveraging kubernetes for adaptive and cost-efficient resource management,” in *Proceedings of the 5th International Workshop on Container Technologies and Container Clouds*, 2019, pp. 37–42.
- [20] G. Rattihalli, M. Govindaraju, H. Lu, and D. Tiwari, “Exploring potential for non-disruptive vertical auto scaling and resource estimation in kubernetes,” in *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*, 2019, pp. 33–40. DOI: [10.1109/CLOUD.2019.00018](https://doi.org/10.1109/CLOUD.2019.00018).
- [21] Z. Xu, Y. Gong, Y. Zhou, Q. Bao, and W. Qian, “Enhancing kubernetes automated scheduling with deep learning and reinforcement techniques for large-scale cloud computing optimization,” in *Ninth International Symposium on Advances in Electrical, Electronics, and Computer Engineering (ISAECE 2024)*, SPIE, vol. 13291, 2024, pp. 1595–1600.
- [22] D.-D. Vu, M.-N. Tran, and Y. Kim, “Predictive hybrid autoscaling for containerized applications,” *IEEE Access*, vol. 10, pp. 109 768–109 778, 2022.
- [23] J. Dogani, F. Khunjush, and M. Seydali, “K-agrued: A container autoscaling technique for cloud-based web applications in kubernetes using attention-based gru encoder-decoder,” *Journal of Grid Computing*, vol. 20, no. 4, p. 40, 2022.
- [24] S. Shim, A. Dhokariya, D. Doshi, S. Upadhye, V. Patwari, and J.-Y. Park, “Predictive auto-scaler for kubernetes cloud,” in *2023 IEEE International Systems Conference (SysCon)*, IEEE, 2023, pp. 1–8.
- [25] S. Chaudhari *et al.*, “An attentive survey of attention models,” *ACM Computing Surveys*, vol. 54, no. 7, pp. 1–32, 2021.
- [26] A. Ba, P. Harsha, and C. Subramanian, “Leveraging interpretability in the transformer to automate the proactive scaling of cloud resources,” in *AAAI Conference on Artificial Intelligence*, 2025.
- [27] H. Luo, Y. Zheng, K. Chen, and S. Zhao, “Probabilistic temporal fusion transformers for large-scale kpi anomaly detection,” *IEEE Access*, vol. 12, pp. 9123–9137, 2024.